



PROMPT DEFINITIVO PARA CLAUDE CODER - PAPERFLOW API 2.0



CONTEXTO E MISSÃO

Você é um **Arquiteto de Software Sênior e Engenheiro Full-Stack** especialista em TypeScript, Fastify, React, PostgreSQL com pgvector, e construção de sistemas enterprise-grade com IA.

Projeto Atual

- **Nome:** PaperFlow Studio
- **Estrutura:** Monorepo (pnpm workspaces)
 - `paperflow-api/`: Backend Fastify + TypeScript
 - `apps/web/`: Frontend React + Vite + TypeScript + Tailwind + shadcn/ui
 - `packages/sdk/`: SDK TypeScript compartilhado

Sua Missão

Executar uma evolução completa do sistema em 3 fases sequenciais, transformando o PaperFlow em uma solução enterprise-ready para o setor jurídico, com foco em:

1. **Estabilização e Correções Críticas**
2. **Arquitetura e Segurança Enterprise**
3. **Features Jurídicas e UI Moderna 2025**



PRINCÍPIOS FUNDAMENTAIS

1. **Código Completo:** Sempre forneça arquivos completos com caminhos absolutos, prontos para copy/paste
2. **Local-First:** Tudo deve funcionar localmente (Docker Compose com Postgres, Redis, MinIO)
3. **Alternância Mock/Real:** Sistema configurável via `USE MOCKS` para desenvolvimento e produção
4. **Segurança Jurídica:** Cadeia de custódia, audit logs, hashing SHA-256, HMAC em webhooks
5. **TypeScript Estrito:** Validação com Zod em todos os pontos de entrada
6. **Observabilidade:** Logs estruturados, métricas Prometheus, tracing
7. **UI Moderna 2025:** Dark-first, OKLCH, glassmorphism sutil, microinterações



FASE 1: CORREÇÕES CRÍTICAS E ESTABILIZAÇÃO

1.1 Correção do SSE (Server-Sent Events)

Arquivo: `paperflow-api/src/routes/documents.ts`

Implementar SSE compatível com EventSource nativo:

- Suporte a autenticação via query param `?apiKey=` e header `x-api-key`
- Heartbeat a cada 15 segundos
- Suporte a reconexão com `Last-Event-ID`
- Formato correto: `id:`, `event:`, `data:`, `retry:`

1.2 Configuração de Ambiente Robusta

Arquivo: `paperflow-api/src/config/env.ts`

typescript

```
import { z } from 'zod';

const envSchema = z.object({
  // Core
  NODE_ENV: z.enum(['development', 'production', 'test']).default('development'),
  PORT: z.coerce.number().default(3002),
  HOST: z.string().default('0.0.0.0'),

  // Feature Flags
  USE MOCKS: z.coerce.boolean().default(true),

  // Database
  DATABASE_URL: z.string().url(),
  DB_SSL: z.coerce.boolean().default(false),

  // Redis
  REDIS_URL: z.string().url(),

  // Storage
  S3_ENDPOINT: z.string().url(),
  S3_ACCESS_KEY: z.string(),
  S3_SECRET_KEY: z.string(),
  S3_BUCKET: z.string().default('paperflow-dev'),
  S3_REGION: z.string().default('us-east-1'),

  // OpenAI
  OPENAI_API_KEY: z.string().startsWith('sk-').optional(),
  OPENAI_MODEL: z.string().default('gpt-4-turbo-preview'),

  // Security
  JWT_SECRET: z.string().min(32),
  API_KEY_SECRET: z.string().min(32),
  WEBHOOK_SECRET: z.string().min(32),

  // Limits
  MAX_FILE_SIZE_MB: z.coerce.number().default(50),
  MAX_PAGES_PER_PDF: z.coerce.number().default(1000),
  RATE_LIMIT_MAX: z.coerce.number().default(100),

  // Features
  ENABLE_OCR: z.coerce.boolean().default(true),
  ENABLE_WEBHOOKS: z.coerce.boolean().default(true),
  ENABLE_METRICS: z.coerce.boolean().default(true),
});
```

```
export const env = envSchema.parse(process.env);
```

1.3 Docker Compose para Desenvolvimento

Arquivo: `paperflow-api/docker-compose.yml`

yml

version: '3.9'

services:

postgres:

image: pgvector/pgvector:pg16

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

POSTGRES_DB: paperflow

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./migrations:/docker-entrypoint-initdb.d

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 10s

timeout: 5s

retries: 5

redis:

image: redis:7-alpine

ports:

- "6379:6379"

volumes:

- redis_data:/data

healthcheck:

test: ["CMD", "redis-cli", "ping"]

interval: 10s

timeout: 5s

retries: 5

minio:

image: minio/minio:latest

command: server /data --console-address ":9001"

environment:

MINIO_ROOT_USER: paperflow

MINIO_ROOT_PASSWORD: paperflow123

ports:

- "9000:9000"

- "9001:9001"

volumes:

- minio_data:/data

healthcheck:

test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]

interval: 10s

timeout: 5s

retries: 5

volumes:

postgres_data:

redis_data:

minio_data:

1.4 Migrations Completas

Arquivo: `paperflow-api/migrations/001_complete_schema.sql`

sql

-- Extensões necessárias

CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE EXTENSION IF NOT EXISTS vector;

CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Tabela de usuários

```
CREATE TABLE IF NOT EXISTS users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email TEXT UNIQUE NOT NULL,  
  company TEXT NOT NULL,  
  plan TEXT NOT NULL CHECK (plan IN ('free', 'pro', 'enterprise')),  
  api_key_hash TEXT NOT NULL,  
  api_secret_hash TEXT NOT NULL,  
  credits_remaining INTEGER DEFAULT 1000,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW(),  
  last_active TIMESTAMPTZ,  
  metadata JSONB DEFAULT '{}::jsonb'  
);
```

-- Tabela de documentos com campos jurídicos

```
CREATE TABLE IF NOT EXISTS documents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  tenant_id TEXT NOT NULL DEFAULT 'default',  
  original_name TEXT NOT NULL,  
  storage_key TEXT NOT NULL,  
  size_bytes BIGINT NOT NULL,  
  mime_type TEXT NOT NULL,  
  sha256_hash TEXT NOT NULL,  
  status TEXT NOT NULL DEFAULT 'pending',  
  pages INTEGER,  
  processing_time_ms INTEGER,  
  error_message TEXT,
```

-- Campos jurídicos

```
case_number TEXT,  
court TEXT,  
parties JSONB,  
document_type TEXT,  
bates_prefix TEXT,  
bates_start INTEGER,  
bates_end INTEGER,
```

```
created_at TIMESTAMPTZ DEFAULT NOW(),  
processed_at TIMESTAMPTZ,
```

```
expires_at TIMESTAMPTZ,  
metadata JSONB DEFAULT '{}::jsonb,  
  
CONSTRAINT valid_status CHECK (status IN ('pending', 'processing', 'completed', 'failed'))  
);
```

-- Tabela de páginas

```
CREATE TABLE IF NOT EXISTS document_pages (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,  
  page_number INTEGER NOT NULL,  
  text TEXT,  
  ocr_confidence FLOAT,  
  word_count INTEGER,  
  char_count INTEGER,  
  checksum TEXT,  
  bates_number TEXT,  
  redactions JSONB DEFAULT '[]::jsonb,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  
  UNIQUE(document_id, page_number)  
);
```

-- Tabela de chunks para RAG

```
CREATE TABLE IF NOT EXISTS chunks (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,  
  page_number INTEGER NOT NULL,  
  chunk_index INTEGER NOT NULL,  
  text TEXT NOT NULL,  
  embedding vector(384),  
  token_count INTEGER,  
  metadata JSONB DEFAULT '{}::jsonb,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  
  UNIQUE(document_id, page_number, chunk_index)  
);
```

-- Tabela de audit log para cadeia de custódia

```
CREATE TABLE IF NOT EXISTS audit_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  document_id UUID REFERENCES documents(id) ON DELETE CASCADE,  
  user_id UUID REFERENCES users(id) ON DELETE SET NULL,  
  actor TEXT NOT NULL,  
  action TEXT NOT NULL,  
  ip_address INET,  
  user_agent TEXT,
```



```
payload JSONB DEFAULT '{}';jsonb,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Tabela de webhooks

```
CREATE TABLE IF NOT EXISTS webhooks (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  url TEXT NOT NULL,  
  events TEXT[] NOT NULL,  
  secret TEXT NOT NULL,  
  active BOOLEAN DEFAULT true,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Tabela de tentativas de webhook

```
CREATE TABLE IF NOT EXISTS webhook_attempts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  webhook_id UUID NOT NULL REFERENCES webhooks(id) ON DELETE CASCADE,  
  event TEXT NOT NULL,  
  payload JSONB NOT NULL,  
  response_status INTEGER,  
  response_body TEXT,  
  error_message TEXT,  
  attempt_number INTEGER DEFAULT 1,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Tabela de pacotes de evidência

```
CREATE TABLE IF NOT EXISTS evidence_packages (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,  
  manifest JSONB NOT NULL,  
  files TEXT[] NOT NULL,  
  package_hash TEXT NOT NULL,  
  created_by UUID REFERENCES users(id),  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Índices para performance

```
CREATE INDEX idx_documents_user_id ON documents(user_id);  
CREATE INDEX idx_documents_status ON documents(status);  
CREATE INDEX idx_documents_sha256 ON documents(sha256_hash);  
CREATE INDEX idx_chunks_document_id ON chunks(document_id);  
CREATE INDEX idx_chunks_embedding ON chunks USING ivfflat (embedding vector_cosine_ops);  
CREATE INDEX idx_audit_log_document ON audit_log(document_id);
```

```
CREATE INDEX idx_audit_log_created ON audit_log(created_at);
CREATE INDEX idx_webhook_attempts_webhook ON webhook_attempts(webhook_id);
```

-- Triggers para updated_at

```
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    NEW.updated_at = NOW();
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER update_users_updated_at BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

```
CREATE TRIGGER update_webhooks_updated_at BEFORE UPDATE ON webhooks
FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

1.5 Correções de Segurança e Error Handling

Arquivo: `paperflow-api/src/app.ts`

Adicionar após registrar plugins:

```
typescript
```

```

// Redação de headers sensíveis
app.addHook('onRequest', async (req) => {
  if (req.headers['x-api-key']) {
    req.headers['x-api-key'] = '[REDACTED]';
  }
  if ((req.query as any)?.apiKey) {
    (req.query as any).apiKey = '[REDACTED]';
  }
});

// Error handler centralizado
app.setErrorHandler((err, req, reply) => {
  const traceId = crypto.randomUUID();
  const status = (err as any).statusCode || 500;

  // Log completo no servidor
  req.log.error({
    err,
    traceId,
    url: req.url,
    method: req.method,
    ip: req.ip,
  });

  // Resposta sanitizada para o cliente
  reply.status(status).send({
    error: {
      code: (err as any).code || 'INTERNAL_SERVER_ERROR',
      message: status < 500 ? err.message : 'An unexpected error occurred',
      traceId,
      ...(env.NODE_ENV !== 'production' && { stack: err.stack }),
    }
  });
});

```

FASE 2: ARQUITETURA ENTERPRISE E FEATURES CORE

2.1 Centralização de Schemas (SDK)

Arquivo: `packages/sdk/src/schemas/document.schema.ts`

typescript

```
import { z } from 'zod';

export const DocumentStatusSchema = z.enum(['pending', 'processing', 'completed', 'failed']);

export const DocumentTypeSchema = z.enum([
  'contract', 'petition', 'motion', 'brief', 'order',
  'judgment', 'deposition', 'discovery', 'correspondence', 'other'
]);

export const PartySchema = z.object({
  name: z.string(),
  role: z.enum(['plaintiff', 'defendant', 'witness', 'attorney', 'judge', 'other']),
  oab: z.string().optional(),
  cpf_cnpj: z.string().optional(),
});

export const DocumentMetadataSchema = z.object({
  title: z.string().optional(),
  author: z.string().optional(),
  creator: z.string().optional(),
  producer: z.string().optional(),
  creationDate: z.string().datetime().optional(),
  modificationDate: z.string().datetime().optional(),
  keywords: z.array(z.string()).optional(),

  // Jurídico
  caseNumber: z.string().optional(),
  court: z.string().optional(),
  judge: z.string().optional(),
  parties: z.array(PartySchema).optional(),
  documentType: DocumentTypeSchema.optional(),

  // Processamento
  wordCount: z.number().optional(),
  pageCount: z.number().optional(),
  hasOcr: z.boolean().optional(),
  ocrConfidence: z.number().min(0).max(1).optional(),
  extractedTables: z.number().optional(),
  detectedPii: z.array(z.string()).optional(),
});

export const DocumentSchema = z.object({
  id: z.string().uuid(),
  userId: z.string().uuid(),
  tenantId: z.string().default('default'),
  originalName: z.string(),
});
```

```

storageKey: z.string(),
sizeBytes: z.number(),
mimeType: z.string(),
sha256Hash: z.string(),
status: DocumentStatusSchema,
pages: z.number().optional(),
processingTimeMs: z.number().optional(),
errorMessage: z.string().optional(),

// Bates numbering
batesPrefix: z.string().optional(),
batesStart: z.number().optional(),
batesEnd: z.number().optional(),

metadata: DocumentMetadataSchema,
createdAt: z.string().datetime(),
processedAt: z.string().datetime().optional(),
expiresAt: z.string().datetime().optional(),
});

export type Document = z.infer<typeof DocumentSchema>;
export type DocumentStatus = z.infer<typeof DocumentStatusSchema>;
export type DocumentMetadata = z.infer<typeof DocumentMetadataSchema>;

```

2.2 Serviço de Cadeia de Custódia

Arquivo: `paperflow-api/src/services/custody-chain.ts`

typescript

```
import crypto from 'crypto';
import { db } from '@config/database';
import { storage } from '@config/storage';

export class CustodyChainService {
  /**
   * Calcula hash SHA-256 de um buffer
   */
  static calculateHash(buffer: Buffer): string {
    return crypto.createHash('sha256').update(buffer).digest('hex');
  }

  /**
   * Gera manifest de evidência com árvore de hashes
   */
  static async generateManifest(documentId: string): Promise<{
    version: string;
    timestamp: string;
    document: {
      id: string;
      hash: string;
      size: number;
      pages: Array<{
        number: number;
        textHash: string;
        checksum: string;
      }>;
      chunks: Array<{
        id: string;
        pageNumber: number;
        index: number;
        textHash: string;
      }>;
    };
    chain: Array<{
      timestamp: string;
      action: string;
      actor: string;
      hash: string;
    }>;
  }> {
    // Buscar documento
    const doc = await db.query(
      'SELECT * FROM documents WHERE id = $1',
      [documentId]
    );
  }
```

// Buscar páginas

```
const pages = await db.query(  
  'SELECT * FROM document_pages WHERE document_id = $1 ORDER BY page_number',  
  [documentId]  
);
```

// Buscar chunks

```
const chunks = await db.query(  
  'SELECT id, page_number, chunk_index, text FROM chunks WHERE document_id = $1',  
  [documentId]  
);
```

// Buscar audit log

```
const auditLog = await db.query(  
  'SELECT * FROM audit_log WHERE document_id = $1 ORDER BY created_at',  
  [documentId]  
);
```

// Construir manifest

```
const manifest = {  
  version: '1.0.0',  
  timestamp: new Date().toISOString(),  
  document: {  
    id: documentId,  
    hash: doc.rows[0].sha256_hash,  
    size: doc.rows[0].size_bytes,  
    pages: pages.rows.map(p => ({  
      number: p.page_number,  
      textHash: this.calculateHash(Buffer.from(p.text || '')),  
      checksum: p.checksum,  
    })),  
    chunks: chunks.rows.map(c => ({  
      id: c.id,  
      pageNumber: c.page_number,  
      index: c.chunk_index,  
      textHash: this.calculateHash(Buffer.from(c.text)),  
    })),  
  },  
  chain: auditLog.rows.map(log => ({  
    timestamp: log.created_at,  
    action: log.action,  
    actor: log.actor,  
    hash: this.calculateHash(Buffer.from(JSON.stringify(log.payload))),  
  })),  
};
```

```

// Salvar manifest
await db.query(
  `INSERT INTO evidence_packages (document_id, manifest, package_hash, files) VALUES ($1, $2, $3, $4)',
  [documentId, JSON.stringify(manifest), this.calculateHash(Buffer.from(JSON.stringify(manifest))), []]
);

return manifest;
}

/**
 * Registra ação no audit log
 */
static async logAction(params: {
  documentId?: string;
  userId?: string;
  action: string;
  actor: string;
  payload?: any;
  ipAddress?: string;
  userAgent?: string;
}): Promise<void> {
  await db.query(
    `INSERT INTO audit_log
    (document_id, user_id, action, actor, payload, ip_address, user_agent)
    VALUES ($1, $2, $3, $4, $5, $6, $7)`,
    [params.documentId, params.userId, params.action, params.actor,
    JSON.stringify(params.payload || {}), params.ipAddress, params.userAgent]
  );
}
}

```

2.3 Serviço de Bates Numbering

Arquivo: `paperflow-api/src/services/bates.ts`

typescript


```
import { PDFDocument, rgb, StandardFonts } from 'pdf-lib';
import { storage } from '@config/storage';
import { db } from '@config/database';
import { CustodyChainService } from './custody-chain';

export interface BatesOptions {
  prefix: string;
  startNumber: number;
  position?: 'top-left' | 'top-right' | 'bottom-left' | 'bottom-right';
  fontSize?: number;
  opacity?: number;
  color?: { r: number; g: number; b: number };
}

export class BatesNumberingService {
  static async applyBatesNumbers(
    documentId: string,
    options: BatesOptions
  ): Promise<{
    success: boolean;
    totalPages: number;
    startNumber: number;
    endNumber: number;
    outputKey: string;
  }> {
    try {
      // Configurações padrão
      const config = {
        position: options.position || 'bottom-right',
        fontSize: options.fontSize || 10,
        opacity: options.opacity || 1.0,
        color: options.color || { r: 0, g: 0, b: 0 },
      };

      // Buscar documento
      const doc = await db.query(
        'SELECT * FROM documents WHERE id = $1',
        [documentId]
      );

      if (!doc.rows.length) {
        throw new Error('Document not found');
      }

      // Baixar PDF original
      const originalPdf = await storage.getObject(doc.rows[0].storage_key);
```

```
const pdfDoc = await PDFDocument.load(originalPdf);
const pages = pdfDoc.getPages();
const font = await pdfDoc.embedFont(StandardFonts.Helvetica);

// Aplicar numeração
let currentNumber = options.startNumber;

for (const page of pages) {
  const { width, height } = page.getSize();
  const batesNumber = `${options.prefix}${String(currentNumber).padStart(6, '0')}`;

  // Calcular posição
  let x = 0, y = 0;
  switch (config.position) {
    case 'top-left':
      x = 50;
      y = height - 30;
      break;
    case 'top-right':
      x = width - 150;
      y = height - 30;
      break;
    case 'bottom-left':
      x = 50;
      y = 30;
      break;
    case 'bottom-right':
      x = width - 150;
      y = 30;
      break;
  }

  // Adicionar texto
  page.drawText(batesNumber, {
    x,
    y,
    size: config.fontSize,
    font,
    color: rgb(config.color.r, config.color.g, config.color.b),
    opacity: config.opacity,
  });

  // Atualizar página no banco
  await db.query(
    'UPDATE document_pages SET bates_number = $1 WHERE document_id = $2 AND page_number = $3',
    [batesNumber, documentId, pages.indexOf(page) + 1]
  );
}
```

```
    currentNumber++;
}

// Salvar PDF com Bates
const pdfBytes = await pdfDoc.save();
const outputKey = `documents/${documentId}/bates.pdf`;
await storage.putObject(outputKey, Buffer.from(pdfBytes));

// Atualizar documento
const endNumber = currentNumber - 1;
await db.query(
  'UPDATE documents SET bates_prefix = $1, bates_start = $2, bates_end = $3 WHERE id = $4',
  [options.prefix, options.startNumber, endNumber, documentId]
);

// Registrar no audit log
await CustodyChainService.logAction({
  documentId,
  action: 'bates_applied',
  actor: 'system',
  payload: {
    prefix: options.prefix,
    startNumber: options.startNumber,
    endNumber,
    totalPages: pages.length,
  },
});

return {
  success: true,
  totalPages: pages.length,
  startNumber: options.startNumber,
  endNumber,
  outputKey,
};
} catch (error) {
  await CustodyChainService.logAction({
    documentId,
    action: 'bates_failed',
    actor: 'system',
    payload: { error: error.message },
  });
  throw error;
}
```

}

}

2.4 Serviço de Detecção e Redação de PII

Arquivo: `paperflow-api/src/services/pii-detector.ts`

typescript

```
export interface PIIMatch {  
  type: string;  
  value: string;  
  start: number;  
  end: number;  
  confidence: number;  
}
```

```
export class PIIDetectorService {  
  private static patterns = {  
    cpf: {  
      regex: /\d{3}\.\?\d{3}\.\?\d{3}-?\d{2}/g,  
      type: 'CPF',  
      confidence: 0.95,  
    },  
    cnpj: {  
      regex: /\d{2}\.\?\d{3}\.\?\d{3}\V?\d{4}-?\d{2}/g,  
      type: 'CNPJ',  
      confidence: 0.95,  
    },  
    oab: {  
      regex: /OAB[\s-]?[A-Z]{2}[\s-]?\d{4,6}/gi,  
      type: 'OAB',  
      confidence: 0.90,  
    },  
    processo_cnj: {  
      regex: /\d{7}-\d{2}\.\d{4}\.\d{1}\.\d{2}\.\d{4}/g,  
      type: 'Processo CNJ',  
      confidence: 0.95,  
    },  
    rg: {  
      regex: /RG[\s:]*\d{1,2}\.\?\d{3}\.\?\d{3}-?\[dXx]/gi,  
      type: 'RG',  
      confidence: 0.85,  
    },  
    telefone: {  
      regex: /\(\?\d{2}\)\?[\s-]?\d{4,5}[-\s]?\d{4}/g,  
      type: 'Telefone',  
      confidence: 0.80,  
    },  
    email: {  
      regex: /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g,  
      type: 'Email',  
      confidence: 0.95,  
    },  
    cep: {
```

```

    regex: /\d{5}-?\d{3}/g,
    type: 'CEP',
    confidence: 0.85,
  },
  placa_veiculo: {
    regex: /[A-Z]{3}[-\s]?\d{4}|[A-Z]{3}[-\s]?\d{1}[A-Z]{1}\d{2}/g,
    type: 'Placa',
    confidence: 0.90,
  },
  conta_bancaria: {
    regex: /(?:contal|c|ag[eê]ncia)[\s:]*\d{3,6}[-\s]?\d{1,2}?/gi,
    type: 'Conta Bancária',
    confidence: 0.75,
  },
};

```

```

/**
 * Detecta PII em um texto
 */
static detectPII(text: string): PIIMatch[] {
  const matches: PIIMatch[] = [];

  for (const [key, pattern] of Object.entries(this.patterns)) {
    const regex = new RegExp(pattern.regex);
    let match;

    while ((match = regex.exec(text)) !== null) {
      matches.push({
        type: pattern.type,
        value: match[0],
        start: match.index,
        end: match.index + match[0].length,
        confidence: pattern.confidence,
      });
    }
  }

  // Ordenar por posição
  return matches.sort((a, b) => a.start - b.start);
}

```

```

/**
 * Redige PII em um texto
 */
static redactText(text: string, matches: PIIMatch[]): string {
  let redactedText = text;

```

// Aplicar redações de trás para frente para manter índices

```
const sortedMatches = [...matches].sort((a, b) => b.start - a.start);
```

```
for (const match of sortedMatches) {  
  const redaction = `[${match.type}: REDACTED]`;   
  redactedText =  
    redactedText.substring(0, match.start) +  
    redaction +  
    redactedText.substring(match.end);  
}
```

```
return redactedText;  
}
```

```
/**
```

```
 * Analisa documento completo para PII
```

```
 */
```

```
static async analyzeDocument(documentId: string): Promise<{
```

```
  totalMatches: number;
```

```
  byType: Record<string, number>;
```

```
  byPage: Array<{
```

```
    pageNumber: number;
```

```
    matches: PIIMatch[];
```

```
  }>;
```

```
  const pages = await db.query(  
    'SELECT page_number, text FROM document_pages WHERE document_id = $1 ORDER BY page_number',  
    [documentId]
```

```
  );
```

```
  const result = {
```

```
    totalMatches: 0,
```

```
    byType: {} as Record<string, number>,
```

```
    byPage: [] as Array<{ pageNumber: number; matches: PIIMatch[] }>,
```

```
  };  
  
  for (const page of pages.rows) {  
    const matches = this.detectPII(page.text || "");  
  
    result.byPage.push(  
      {  
        pageNumber: page.page_number,  
        matches,  
      });  
  
    result.totalMatches += matches.length;  
  
    for (const match of matches) {
```

```

        result.byType[match.type] = (result.byType[match.type] || 0) + 1;
    }
}

// Salvar resultados nos metadados
await db.query(
    `UPDATE documents
    SET metadata = jsonb_set(metadata, '{detectedPii}', $1::jsonb)
    WHERE id = $2`,
    [JSON.stringify(Object.keys(result.byType)), documentId]
);

// Registrar no audit log
await CustodyChainService.logAction({
    documentId,
    action: 'pii_analysis_completed',
    actor: 'system',
    payload: {
        totalMatches: result.totalMatches,
        types: Object.keys(result.byType),
    },
});

return result;
}
}

```

2.5 Webhook Service com HMAC

Arquivo: `paperflow-api/src/services/webhook.ts`

typescript


```
import crypto from 'crypto';
import { db } from '@config/database';
import { env } from '@config/env';

export interface WebhookPayload {
  event: string;
  timestamp: string;
  data: any;
}

export class WebhookService {
  /**
   * Gera assinatura HMAC para o payload
   */
  private static generateSignature(payload: string, secret: string): string {
    return crypto
      .createHmac('sha256', secret)
      .update(payload)
      .digest('hex');
  }

  /**
   * Verifica se o timestamp não é muito antigo (anti-replay)
   */
  private static isTimestampValid(timestamp: string, maxAgeMs = 300000): boolean {
    const age = Date.now() - new Date(timestamp).getTime();
    return age >= 0 && age <= maxAgeMs;
  }

  /**
   * Envia webhook com retry e backoff
   */
  static async send(
    webhookId: string,
    event: string,
    data: any,
    attemptNumber = 1
  ): Promise<void> {
    const maxAttempts = 3;

    try {
      // Buscar webhook
      const webhook = await db.query(
        'SELECT * FROM webhooks WHERE id = $1 AND active = true',
        [webhookId]
      );
    }
  }
}
```

```
if (!webhook.rows.length) {
  throw new Error('Webhook not found or inactive');
}

const { url, secret, events } = webhook.rows[0];

// Verificar se o evento está configurado
if (!events.includes(event)) {
  return;
}

// Preparar payload
const timestamp = new Date().toISOString();
const payload: WebhookPayload = {
  event,
  timestamp,
  data,
};

const payloadString = JSON.stringify(payload);
const signature = this.generateSignature(payloadString, secret);

// Enviar webhook
const response = await fetch(url, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-PF-Event': event,
    'X-PF-Signature': signature,
    'X-PF-Timestamp': timestamp,
    'X-PF-Version': '1.0',
  },
  body: payloadString,
  signal: AbortSignal.timeout(10000), // 10s timeout
});

// Registrar tentativa
await db.query(
  `INSERT INTO webhook_attempts
  (webhook_id, event, payload, response_status, response_body, attempt_number)
  VALUES ($1, $2, $3, $4, $5, $6)`,
  [webhookId, event, payload, response.status, await response.text(), attemptNumber]
);

if (!response.ok && attemptNumber < maxAttempts) {
  // Retry com backoff exponencial
}
```

```

const delay = Math.pow(2, attemptNumber) * 1000;
setTimeout(() => {
  this.send(webhookId, event, data, attemptNumber + 1);
}, delay);
}
} catch (error) {
  // Registrar erro
  await db.query(
    `INSERT INTO webhook_attempts
      (webhook_id, event, payload, error_message, attempt_number)
      VALUES ($1, $2, $3, $4, $5)`,
    [webhookId, event, { data }, error.message, attemptNumber]
  );

  if (attemptNumber < maxAttempts) {
    const delay = Math.pow(2, attemptNumber) * 1000;
    setTimeout(() => {
      this.send(webhookId, event, data, attemptNumber + 1);
    }, delay);
  }
}
}

/**
 * Dispara webhooks para todos os usuários inscritos
 */
static async trigger(userId: string, event: string, data: any): Promise<void> {
  const webhooks = await db.query(
    'SELECT id FROM webhooks WHERE user_id = $1 AND $2 = ANY(events) AND active = true',
    [userId, event]
  );

  for (const webhook of webhooks.rows) {
    // Processar de forma assíncrona
    this.send(webhook.id, event, data).catch(console.error);
  }
}

/**
 * Verifica assinatura de webhook recebido (para documentação)
 */
static verifySignature(
  payload: string,
  signature: string,
  secret: string,
  timestamp: string
): boolean {

```

```
if (!this.isTimestampValid(timestamp)) {  
  return false;  
}  
  
const expectedSignature = this.generateSignature(payload, secret);  
return crypto.timingSafeEqual(  
  Buffer.from(signature),  
  Buffer.from(expectedSignature)  
);  
}  
}
```

2.6 Métricas Prometheus

Arquivo: `paperflow-api/src/plugins/metrics.ts`

typescript

```
import { FastifyPluginAsync } from 'fastify';
import { register, collectDefaultMetrics, Counter, Histogram, Gauge } from 'prom-client';

// Registrar métricas padrão
collectDefaultMetrics();

// Métricas customizadas
const httpDuration = new Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status'],
  buckets: [0.1, 0.3, 0.5, 0.7, 1, 3, 5, 7, 10],
});

const documentsProcessed = new Counter({
  name: 'paperflow_documents_processed_total',
  help: 'Total number of documents processed',
  labelNames: ['status'],
});

const pipelineStage = new Histogram({
  name: 'paperflow_pipeline_stage_duration_seconds',
  help: 'Duration of pipeline stages in seconds',
  labelNames: ['stage'],
  buckets: [0.5, 1, 2, 5, 10, 30, 60],
});

const activeSSEConnections = new Gauge({
  name: 'paperflow_sse_connections_active',
  help: 'Number of active SSE connections',
});

const piiDetected = new Counter({
  name: 'paperflow_pii_detected_total',
  help: 'Total PII entities detected',
  labelNames: ['type'],
});

export const metrics = {
  httpDuration,
  documentsProcessed,
  pipelineStage,
  activeSSEConnections,
  piiDetected,
};
```

```
const metricsPlugin: FastifyPluginAsync = async (fastify) => {  
  // Hook para medir duração das requisições  
  fastify.addHook('onRequest', async (req, reply) => {  
    (reply as any).startTime = Date.now();  
  });  
  
  fastify.addHook('onResponse', async (req, reply) => {  
    const duration = (Date.now() - (reply as any).startTime) / 1000;  
    httpDuration  
      .labels(req.method, req.routerPath || req.url, reply.statusCode.toString())  
      .observe(duration);  
  });  
  
  // Endpoint de métricas  
  fastify.get('/v1/metrics', async (req, reply) => {  
    reply.type('text/plain');  
    return register.metrics();  
  });  
};  
  
export default metricsPlugin;
```



FASE 3: UI MODERNA 2025 E FEATURES JURÍDICAS

3.1 Design System com OKLCH

Arquivo: `apps/web/src/styles/tokens.css`

CSS

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400..700&display=swap');
```

```
:root {  
  /* Cores em OKLCH para melhor percepção */  
  --color-bg: oklch(10% 0.02 250);  
  --color-surface: oklch(15% 0.02 250);  
  --color-surface-elevated: oklch(18% 0.02 250);  
  
  --color-accent: oklch(65% 0.3 250);  
  --color-accent-hover: oklch(70% 0.35 250);  
  --color-accent-contrast: oklch(98% 0.02 250);  
  
  --color-success: oklch(70% 0.2 145);  
  --color-warning: oklch(75% 0.2 70);  
  --color-error: oklch(65% 0.3 25);  
  
  --color-text: oklch(95% 0.02 250);  
  --color-text-muted: oklch(70% 0.02 250);  
  
  /* Espaçamento e bordas */  
  --radius-sm: 6px;  
  --radius-md: 12px;  
  --radius-lg: 16px;  
  --radius-xl: 20px;  
  
  /* Efeitos */  
  --blur-sm: 8px;  
  --blur-md: 12px;  
  --blur-lg: 20px;  
  
  --shadow-sm: 0 2px 8px oklch(0% 0 0 / 0.1);  
  --shadow-md: 0 4px 16px oklch(0% 0 0 / 0.15);  
  --shadow-lg: 0 8px 32px oklch(0% 0 0 / 0.2);  
  
  /* Gradientes */  
  --gradient-radial: radial-gradient(  
    circle at 30% 20%,  
    oklch(20% 0.1 250) 0%,  
    oklch(10% 0.02 250) 60%  
  );  
  
  --gradient-accent: linear-gradient(  
    135deg,  
    oklch(65% 0.3 250) 0%,  
    oklch(70% 0.35 280) 100%  
  );  
}
```

```
/* Animação */
--ease-out-cubic: cubic-bezier(0.33, 1, 0.68, 1);
--duration-fast: 180ms;
--duration-normal: 240ms;
--duration-slow: 360ms;
}

/* Glass effect mixin */
.glass {
  background: oklch(15% 0.02 250 / 0.8);
  backdrop-filter: blur(var(--blur-md));
  border: 1px solid oklch(100% 0 0 / 0.1);
}

/* Noise texture overlay */
.noise::before {
  content: "";
  position: absolute;
  inset: 0;
  background-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 256 256' xmlns='http://www.w3.org/2000/svg'%3
  pointer-events: none;
  opacity: 0.5;
}
```

3.2 Componente Document Viewer Jurídico

Arquivo: `apps/web/src/components/DocumentViewer.tsx`

tsx


```

import React, { useState, useEffect } from 'react';
import { Tabs, TabsContent, TabsList, TabsTrigger } from '@components/ui/tabs';
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from '@components/ui/card';
import { Badge } from '@components/ui/badge';
import { Button } from '@components/ui/button';
import { ScrollArea } from '@components/ui/scroll-area';
import { Alert, AlertDescription } from '@components/ui/alert';
import {
  FileText,
  Shield,
  Hash,
  AlertTriangle,
  Calendar,
  Users,
  Gavel,
  Eye,
  EyeOff,
  Download,
  CheckCircle
} from 'lucide-react';
import { useAppStore } from '@stores/app-store';
import { subscribeToDocumentEvents } from '@paperflow/sdk';

interface DocumentViewerProps {
  documentId: string;
}

export function DocumentViewer({ documentId }: DocumentViewerProps) {
  const [document, setDocument] = useState<any>(null);
  const [activeTab, setActiveTab] = useState('events');
  const [sseEvents, setSSEEvents] = useState<any[]>([]);
  const [manifest, setManifest] = useState<any>(null);
  const [piiAnalysis, setPiiAnalysis] = useState<any>(null);
  const [showRedaction, setShowRedaction] = useState(false);
  const { apiKey, apiBaseUrl } = useAppStore();

  // Conectar SSE
  useEffect(() => {
    if (!apiKey || !documentId) return;

    const unsubscribe = subscribeToDocumentEvents(
      documentId,
      apiKey,
      apiBaseUrl,
      {
        onProgress: (data) => {

```

```

    setSSEEvents(prev => [...prev, { type: 'progress', ...data }]);
  },
  onComplete: (data) => {
    setSSEEvents(prev => [...prev, { type: 'complete', ...data }]);
    loadDocument();
  },
  onError: (error) => {
    setSSEEvents(prev => [...prev, { type: 'error', error }]);
  },
}
);

return unsubscribe;
}, [documentId, apiKey]);

// Carregar documento
const loadDocument = async () => {
  const response = await fetch(`${apiBaseUrl}/v1/documents/${documentId}`, {
    headers: { 'x-api-key': apiKey },
  });
  const data = await response.json();
  setDocument(data);

  // Carregar manifest
  const manifestRes = await fetch(`${apiBaseUrl}/v1/documents/${documentId}/manifest`, {
    headers: { 'x-api-key': apiKey },
  });
  if (manifestRes.ok) {
    setManifest(await manifestRes.json());
  }

  // Carregar análise PII
  const piiRes = await fetch(`${apiBaseUrl}/v1/documents/${documentId}/redact/preview`, {
    headers: { 'x-api-key': apiKey },
  });
  if (piiRes.ok) {
    setPiiAnalysis(await piiRes.json());
  }
};

useEffect(() => {
  loadDocument();
}, [documentId]);

const applyBates = async () => {
  const response = await fetch(`${apiBaseUrl}/v1/documents/${documentId}/bates`, {
    method: 'POST',
  });

```

```

headers: {
  'x-api-key': apiKey,
  'Content-Type': 'application/json',
},
body: JSON.stringify({
  prefix: 'CASE-2025-',
  startNumber: 1,
}),
});

if (response.ok) {
  loadDocument();
}
};

const exportEvidence = async () => {
  const response = await fetch(`${apiBaseUrl}/v1/documents/${documentId}/evidence/export`, {
    method: 'POST',
    headers: { 'x-api-key': apiKey },
  });

  if (response.ok) {
    const data = await response.json();
    window.open(data.downloadUrl, '_blank');
  }
};

if (!document) {
  return <div>Carregando...</div>;
}

return (
  <div className="container mx-auto p-6 space-y-6">
    {/* Header */}
    <Card className="glass">
      <CardHeader>
        <div className="flex items-start justify-between">
          <div>
            <CardTitle className="text-2xl flex items-center gap-2">
              <FileText className="h-6 w-6" />
              {document.originalName}
            </CardTitle>
            <CardDescription className="mt-2">
              <div className="flex flex-wrap gap-4 text-sm">
                <span className="flex items-center gap-1">
                  <Hash className="h-4 w-4" />
                  SHA-256: {document.sha256Hash?.substring(0, 12)}...

```

```

</span>
<span className="flex items-center gap-1">
  <Calendar className="h-4 w-4" />
  {new Date(document.createdAt).toLocaleDateString()}
</span>
{document.metadata?.caseNumber && (
  <span className="flex items-center gap-1">
    <Gavel className="h-4 w-4" />
    Processo: {document.metadata.caseNumber}
  </span>
)}
</div>
</CardDescription>
</div>
<div className="flex gap-2">
  <Button onClick={applyBates} variant="outline" size="sm">
    <Hash className="h-4 w-4 mr-2" />
    Aplicar Bates
  </Button>
  <Button onClick={exportEvidence} variant="outline" size="sm">
    <Download className="h-4 w-4 mr-2" />
    Exportar Evidências
  </Button>
</div>
</div>
</CardHeader>
</Card>

<div className="grid grid-cols-12 gap-6">
  {/* Main Content */}
  <div className="col-span-8">
    <Tabs value={activeTab} onValueChange={setActiveTab}>
      <TabsList className="glass w-full">
        <TabsTrigger value="events">Eventos</TabsTrigger>
        <TabsTrigger value="text">Texto</TabsTrigger>
        <TabsTrigger value="tables">Tabelas</TabsTrigger>
        <TabsTrigger value="metadata">Metadados</TabsTrigger>
        <TabsTrigger value="custody">Cadeia de Custódia</TabsTrigger>
      </TabsList>

      <TabsContent value="events" className="mt-4">
        <Card>
          <CardHeader>
            <CardTitle>Eventos de Processamento</CardTitle>
          </CardHeader>
          <CardContent>
            <ScrollArea className="h-[400px]">

```

```

<div className="space-y-2">
  {sseEvents.map((event, i) => (
    <div key={i} className="flex items-center gap-3 p-3 rounded-lg bg-surface">
      {event.type === 'complete' ? (
        <CheckCircle className="h-5 w-5 text-green-500" />
      ) : event.type === 'error' ? (
        <AlertTriangle className="h-5 w-5 text-red-500" />
      ) : (
        <div className="h-5 w-5 rounded-full border-2 border-accent animate-spin" />
      )}
      <div className="flex-1">
        <div className="font-medium">{event.stage || event.type}</div>
        <div className="text-sm text-muted">{event.message}</div>
      </div>
      <div className="text-xs text-muted">
        {new Date(event.timestamp).toLocaleTimeString()}
      </div>
    </div>
  )))
</div>
</ScrollArea>
</CardContent>
</Card>
</TabsContent>

<TabsContent value="text" className="mt-4">
  <Card>
    <CardHeader>
      <CardTitle className="flex items-center justify-between">
        Texto Extraído
        <Button
          variant="ghost"
          size="sm"
          onClick={() => setShowRedaction(!showRedaction)}
        >
          {showRedaction ? <EyeOff className="h-4 w-4" /> : <Eye className="h-4 w-4" />}
          {showRedaction ? 'Ocultar' : 'Mostrar'} Redações
        </Button>
      </CardTitle>
    </CardHeader>
    <CardContent>
      <ScrollArea className="h-[500px]">
        <div className="space-y-4">
          {document.pages?.map((page: any) => (
            <div key={page.pageNumber} className="space-y-2">
              <div className="flex items-center gap-2 sticky top-0 bg-background py-2">
                <Badge variant="outline">Página {page.pageNumber}</Badge>

```

```

        {page.batesNumber && (
          <Badge variant="secondary">{page.batesNumber}</Badge>
        )}
      </div>
      <div className="p-4 rounded-lg bg-surface font-mono text-sm whitespace-pre-wrap">
        {showRedaction && piiAnalysis?.byPage?.find((p: any) => p.pageNumber === page.pageNumber)
          ? <RedactedText
            text={page.text}
            matches={piiAnalysis.byPage.find((p: any) => p.pageNumber === page.pageNumber).matches}
          />
          : page.text
        }
      </div>
    </div>
  </div>
)))
</div>
</ScrollArea>
</CardContent>
</Card>
</TabsContent>

<TabsContent value="custody" className="mt-4">
  <Card>
    <CardHeader>
      <CardTitle className="flex items-center gap-2">
        <Shield className="h-5 w-5" />
        Cadeia de Custódia
      </CardTitle>
    </CardHeader>
    <CardContent>
      {manifest && (
        <div className="space-y-4">
          <Alert>
            <Shield className="h-4 w-4" />
            <AlertDescription>
              Manifest gerado em {new Date(manifest.timestamp).toLocaleString()}
            <br />
              Hash do Pacote: <code className="text-xs">{manifest.packageHash}</code>
            </AlertDescription>
          </Alert>

          <div className="space-y-2">
            <h4 className="font-medium">Cadeia de Eventos:</h4>
            {manifest.chain?.map((event: any, i: number) => (
              <div key={i} className="flex items-center gap-3 p-3 rounded-lg bg-surface">
                <div className="w-2 h-2 rounded-full bg-accent" />
                <div className="flex-1">

```

```

        <div className="font-medium">{event.action}</div>
        <div className="text-sm text-muted">Por: {event.actor}</div>
      </div>
      <div className="text-xs text-muted">
        {new Date(event.timestamp).toLocaleString()}
      </div>
    </div>
  )))
</div>
</div>
  })
</CardContent>
</Card>
</TabsContent>
</Tabs>
</div>

```

{/ Sidebar - Copiloto Jurídico */}*

```

<div className="col-span-4">
  <Card className="glass sticky top-6">
    <CardHeader>
      <CardTitle className="flex items-center gap-2">
        <Gavel className="h-5 w-5" />
        Copiloto Jurídico
      </CardTitle>
      <CardDescription>
        Análise automática do documento
      </CardDescription>
    </CardHeader>
    <CardContent className="space-y-4">
      {/* Detecção de PII */}
      {piiAnalysis && (
        <div className="space-y-2">
          <h4 className="font-medium flex items-center gap-2">
            <AlertTriangle className="h-4 w-4" />
            Dados Sensíveis Detectados
          </h4>
          <div className="space-y-1">
            {Object.entries(piiAnalysis.byType).map(([type, count]: [string, any]) => (
              <div key={type} className="flex items-center justify-between p-2 rounded bg-surface">
                <span className="text-sm">{type}</span>
                <Badge variant="destructive">{count}</Badge>
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
  </div>

```

```

{ /* Partes do Processo */
{document.metadata?.parties && (
  <div className="space-y-2">
    <h4 className="font-medium flex items-center gap-2">
      <Users className="h-4 w-4" />
      Partes Envolvidas
    </h4>
    <div className="space-y-1">
      {document.metadata.parties.map((party: any, i: number) => (
        <div key={i} className="p-2 rounded bg-surface">
          <div className="font-medium text-sm">{party.name}</div>
          <div className="text-xs text-muted">{party.role}</div>
        </div>
      ))}
    </div>
  </div>
)}

```

```

{ /* Prazos Detectados */
<div className="space-y-2">
  <h4 className="font-medium flex items-center gap-2">
    <Calendar className="h-4 w-4" />
    Prazos Processuais
  </h4>
  <Alert>
    <AlertDescription className="text-sm">
      Análise de prazos em desenvolvimento
    </AlertDescription>
  </Alert>
</div>

```

```

{ /* Cláusulas Contratuais */
{document.metadata?.documentType === 'contract' && (
  <div className="space-y-2">
    <h4 className="font-medium">Cláusulas Detectadas</h4>
    <div className="space-y-1">
      {[ 'Confidencialidade', 'Multa', 'Rescisão', 'Foro' ].map(clause => (
        <div key={clause} className="flex items-center justify-between p-2 rounded bg-surface">
          <span className="text-sm">{clause}</span>
          <Badge variant="outline">Detectada</Badge>
        </div>
      ))}
    </div>
  </div>
)}
</CardContent>

```



```

        </Card>
      </div>
    </div>
  </div>
);
}

// Componente auxiliar para texto com redação
function RedactedText({ text, matches }: { text: string; matches: any[] }) {
  let result = [];
  let lastEnd = 0;

  const sortedMatches = [...matches].sort((a, b) => a.start - b.start);

  for (const match of sortedMatches) {
    if (match.start > lastEnd) {
      result.push(text.substring(lastEnd, match.start));
    }
    result.push(
      <span key={match.start} className="bg-black text-black select-none">
        {'█'.repeat(match.value.length)}
      </span>
    );
    lastEnd = match.end;
  }

  if (lastEnd < text.length) {
    result.push(text.substring(lastEnd));
  }

  return <>{result}</>;
}

```

3.3 Command Palette

Arquivo: apps/web/src/components/CommandPalette.tsx

tsx

```
import React, { useEffect, useState } from 'react';
import {
  CommandDialog,
  CommandEmpty,
  CommandGroup,
  CommandInput,
  CommandItem,
  CommandList,
  CommandSeparator,
} from '@components/ui/command';
import {
  Upload,
  Hash,
  Shield,
  Download,
  Search,
  FileText,
  Settings,
  BarChart,
} from 'lucide-react';
import { useNavigate } from 'react-router-dom';

export function CommandPalette() {
  const [open, setOpen] = useState(false);
  const navigate = useNavigate();

  useEffect(() => {
    const down = (e: KeyboardEvent) => {
      if (e.key === 'k' && (e.metaKey || e.ctrlKey)) {
        e.preventDefault();
        setOpen((open) => !open);
      }
    };

    document.addEventListener('keydown', down);
    return () => document.removeEventListener('keydown', down);
  }, []);

  const runCommand = (command: string) => {
    setOpen(false);

    switch (command) {
      case 'upload':
        navigate('/upload');
        break;
      case 'documents':
```

```

    navigate('/documents');
    break;
case 'metrics':
    navigate('/metrics');
    break;
case 'settings':
    navigate('/settings');
    break;
case 'bates':
    // Trigger Bates numbering on current document
    break;
case 'redact':
    // Toggle redaction mode
    break;
case 'export':
    // Export evidence package
    break;
case 'search':
    // Focus search input
    break;
}
};

return (
  <CommandDialog open={open} onOpenChange={setOpen}>
    <CommandInput placeholder="Digite um comando ou pesquise..." />
    <CommandList>
      <CommandEmpty>Nenhum resultado encontrado.</CommandEmpty>

      <CommandGroup heading="Ações">
        <CommandItem onSelect={() => runCommand('upload')}>
          <Upload className="mr-2 h-4 w-4" />
          <span>Upload de Documento</span>
        </CommandItem>
        <CommandItem onSelect={() => runCommand('bates')}>
          <Hash className="mr-2 h-4 w-4" />
          <span>Aplicar Numeração Bates</span>
        </CommandItem>
        <CommandItem onSelect={() => runCommand('redact')}>
          <Shield className="mr-2 h-4 w-4" />
          <span>Modo de Redação</span>
        </CommandItem>
        <CommandItem onSelect={() => runCommand('export')}>
          <Download className="mr-2 h-4 w-4" />
          <span>Exportar Dossiê de Evidências</span>
        </CommandItem>
      </CommandGroup>
    </CommandList>
  </CommandDialog>
);

```

<CommandSeparator />

<C